

Kiểm thử bảo mật bằng Slither (Bước 4, mục 11)

1. Tình trạng cài đặt Slither

Đã cài thành công:

```
python -m pip install --user slither-analyzer
# slither-analyzer 0.11.5, solc 0.8.36 (solc-select)
```

2. Sự cố tương thích đã gặp — không chạy được phân tích tự động

Slither không thể phân tích project này, vì lý do đã xác định rõ (không phải lỗi ở contract):

2.1. `cryptic-compile` chưa hỗ trợ định dạng artifact của Hardhat 3

Chạy `slither .` (chế độ tự nhận diện project Hardhat) báo lỗi:

```
Problem deserializing hardhat configuration, using defaults: Expecting value: line 1 column 1 (char 0)
...
File "cryptic_compile\platform\hardhat.py", line 72, in hardhat_like_parsing
    targets_json = loaded_json["output"]
KeyError: 'output'
```

`npx hardhat compile --force` chạy thành công (project vẫn biên dịch bình thường qua `npx hardhat build / npm test`), nhưng `cryptic-compile` (thư viện `slither` dùng để đọc kết quả biên dịch của Hardhat) mong đợi cấu trúc file build-info theo định dạng **Hardhat 2**, trong khi Hardhat 3 (dùng ở project này theo đúng quyết định "bám sát 100% công cụ trong tài liệu mẫu") đã đổi cấu trúc artifact/build-info. Đây là khoảng trễ hỗ trợ giữa hai công cụ của hai bên phát triển khác nhau, không phải lỗi trong `RentalManager.sol` hay `RentalAgreementToken.sol`.

2.2. Đường vòng qua solc thuần cũng thất bại — do đường dẫn Unicode trên Windows

Thử đường vòng (`--compile-force-framework solc`, gọi solc trực tiếp thay vì qua Hardhat) cũng thất bại, nhưng vì lý do **khác**: bản thân lệnh `solc` (bản `0.8.36+commit.8a079791.Windows.msvc` cài qua `solc-select`) không tự resolve được ngay cả import **tương đối cùng thư mục** (`import "../RentalAgreementToken.sol";`):

```
solc @openzeppelin/=node_modules/@openzeppelin/ contracts/RentalManager.sol --allow-paths .
Error: Source "contracts/RentalAgreementToken.sol" not found: File not found.
```

Đã xác minh đây là lỗi của bản thân `solc.exe` (không phải do `slither / cryptic-compile` gọi sai tham số) bằng cách chạy thẳng lệnh `solc` này độc lập — lỗi giống hệt. Nguyên nhân nhiều khả năng là **đường dẫn thư mục project chứa ký tự tiếng Việt có dấu và khoảng trắng** (`D:\Hệ thống quản lý thuê nhà bằng smart contract`), khiến bản solc native (MSVC) xử lý sai đường dẫn khi tìm file. Hardhat tự biên dịch được vì nó dùng bản **solc WASM** (tải riêng, chạy qua Node.js — Node xử lý Unicode path bình thường), trong khi `solc-select` cài bản solc native cho Windows.

2.3. Hướng khắc phục (chưa thực hiện trong lần này)

- Chờ `cryptic-compile` / `slither` ra bản hỗ trợ đầy đủ định dạng build-info của Hardhat 3, hoặc
- Copy project sang một đường dẫn không dấu/không khoảng trắng (vd `C:\rental-dapp`) rồi chạy lại `slither` từ đó, hoặc
- Cấu hình `solc-select` dùng bản solc WASM thay vì native.

3. Rà soát bảo mật thủ công (thay thế tạm thời cho Slither)

Vì không chạy được Slither tự động, đã rà soát thủ công theo đúng các nhóm lỗi Slither thường phát hiện (reentrancy, access control, unchecked call, zero-address, gas/DoS).

#	Hạng mục	Kết quả	Phân loại
1	Reentrancy ở <code>rentProperty</code> / <code>payRent</code> / <code>acceptSettlement</code> / <code>voteOnDispute</code> (chuyển ETH)	Có <code>nonReentrant</code> + checks-effects-interactions (state cập nhật trước khi gọi <code>.call{value:}</code> hoặc <code>mintAgreement</code>)	Accepted — đã xử lý
2	Giá trị trả về của low-level <code>.call{value:}</code>	Luôn kiểm tra bằng <code>require(ok, ...)</code> , không bỏ qua	Accepted — đã xử lý
3	<code>mintAgreement()</code> gọi trong <code>rentProperty</code> không bọc try/catch	Nếu mint revert (vd <code>RentalManager</code> chưa được cấp <code>MINTER_ROLE</code>), cả giao dịch <code>rentProperty</code> revert theo — đây là hành vi mong muốn (fail-closed, không cho kích hoạt hợp đồng nếu không tạo được token bằng chứng)	False positive — đúng thiết kế
4	Địa chỉ <code>0x0</code> cho <code>tenant</code> khi mint	Không kiểm tra tường minh trong <code>RentalManager</code> , nhưng <code>ERC721._safeMint</code> (OpenZeppelin) tự revert nếu <code>to == address(0)</code>	Accepted — đã có bảo vệ ở tầng dưới
5	<code>getAllProperties()</code> lặp không giới hạn qua toàn bộ <code>propertyCount</code>	Nếu số lượng property rất lớn, có thể tốn nhiều gas khi gọi từ một contract khác (dù gọi trực tiếp qua <code>eth_call</code> từ frontend không bị giới hạn gas theo cách này)	Design concern — đã ghi trong docs/gioi-han-va-rui-ro.md, chưa khắc phục (cần phân trang nếu mở rộng)
6	Không có <code>receive()</code> / <code>fallback()</code>	ETH gửi thẳng vào contract (không gọi hàm) sẽ tự động revert — tránh ETH bị kẹt ngoài ý muốn	Accepted — hành vi an toàn theo mặc định của Solidity

#	Hạng mục	Kết quả	Phân loại
7	Kiểm tra địa chỉ <code>0x0</code> ở constructor	Cả 2 contract đều <code>require / revert</code> nếu <code>admin / tokenAddress</code> là <code>address(0)</code>	Accepted — đã xử lý
8	Double-mint cùng <code>tokenId</code>	Không tự kiểm tra tương minh trong <code>RentalManager</code> , nhưng <code>_safeMint</code> của OpenZeppelin tự revert nếu token đã tồn tại — về logic nghiệp vụ, một <code>propertyId</code> chỉ có thể đạt <code>Status.Active</code> (điều kiện duy nhất gọi mint) đúng một lần trong vòng đời hiện tại	Accepted — không khai thác được với logic hiện tại
9	Tập trung quyền lực ở <code>RentalAgreementToken.DEFAULT_ADMIN_ROLE</code>	Địa chỉ admin (mặc định là ví deploy) có toàn quyền <code>grantRole / revokeRole</code> cho <code>MINTER_ROLE</code> — nếu khoá riêng của admin bị lộ, kẻ tấn công có thể tự cấp quyền mint cho địa chỉ khác và tạo token "hợp đồng thuê" giả (không rút được tiền từ <code>RentalManager</code> , nhưng gây nhiễu dữ liệu lịch sử thuê)	Confirmed — rủi ro tập trung quyền lực, cần multisig cho admin ở production (xem <code>docs/production-checklist.md</code>)
10	Thiếu cơ chế tạm dừng (<code>Pausable</code>)	Không có cách nào tạm dừng hệ thống nếu phát hiện lỗi nghiêm trọng sau khi deploy	Confirmed — hạn chế đã biết, chưa khắc phục trong lần này (xem <code>docs/production-checklist.md</code>)
11	<code>cancellisting()</code> — chủ nhà tự huỷ tin của mình	Chỉ đổi <code>status</code> , không chuyển tiền, không ảnh hưởng tin khác; <code>require(msg.sender == p.landlord) + require(status == Listed)</code> chặn đúng	Accepted — đã xử lý

#	Hạng mục	Kết quả	Phân loại
12	<code>voteOnDispute()</code> không kiểm tra người bỏ phiếu có phải chính chủ nhà/người thuê của tranh chấp đó không — chỉ kiểm tra <code>ARBITER_ROLE</code>	Phát hiện thật trong lúc test (không phải lý thuyết): ví admin (đồng thời là chủ nhà) tự bỏ phiếu được cho tranh chấp của chính mình vì <code>ARBITER_ROLE</code> được cấp mặc định cho admin lúc deploy	Confirmed — xung đột lợi ích, đã giảm thiểu bằng thu hồi/cấp lại vai trò (<code>scripts/revoke-arbiter.ts</code>), CHƯA chặn ở mức contract (xem <code>docs/gioi-han-va-rui-ro.md § 4</code>)

4. Kết luận

Không phát hiện lỗi **Confirmed vulnerability** nghiêm trọng (mất tiền/khoá tiền) qua rà soát thủ công. Ba điểm được đánh dấu **Confirmed** (#9, #10, #12) là rủi ro vận hành/quản trị/thiết kế đã biết trước — trong đó #12 là lỗi xung đột lợi ích thực sự đã xảy ra trong lúc test (không phải suy đoán lý thuyết) — và đã được đưa vào [docs/production-checklist.md](#) / [docs/gioi-han-va-rui-ro.md](#) như điều kiện bắt buộc phải xử lý trước khi triển khai production thật.

Rà soát thủ công **không thay thế được** một audit độc lập hoặc phân tích tĩnh tự động đầy đủ — đây vẫn là việc cần làm trước khi triển khai production (xem checklist).